Implementación de Tetris mediante manipulación de bits en C++

Mateo Arbelaez Cardona y Jean Israel Garces Escobar

Facultad de Ingeniería, Universidad de Antioquia

Informática II

Aníbal Guerra y Augusto Salazar

22 Marzo de 2026

Contextualización

En primera instancia se explicará el problema y el contexto que rodea al desafío, se empezará desde lo más básico y es que hay que hacer, y de allí responder preguntas conceptuales importantes y la demostración del entendimiento del documento proporcionado por el docente.

En pocas palabras el desafío consiste en hacer una versión limitada de Tetris usando lenguaje de programación de C++ usando el IDE Qt.

El principal propósito del proyecto es aplicar conceptos de programación estructurada, uso de operaciones bit a bit, gestión eficiente de la memoria.

Ahora se hará una pequeña salvedad para explicar qué es y en qué consiste el Tetris.

Tetris es un videojuego de lógica creado en 1984 por Alexey Pajitnov. El juego se basa en la caída de piezas geométricas llamadas tetróminos, cada una está formada por cuatro bloques cuadrados. Estas piezas descienden desde la parte superior de un tablero rectangular y el jugador puede moverlas hacia la izquierda, hacia la derecha o rotarlas antes de que se fijen definitivamente. El objetivo del juego es completar líneas horizontales sin dejar espacios vacíos; cuando una línea se completa, esta desaparece y las piezas ubicadas arriba descienden, permitiendo continuar el juego.

Entonces, en el avance del documento, se tiene que considerar cosas relevantes como la representación del tablero, la generación aleatoria de las piezas, la detección de colisiones, el cómo se eliminan las filas completas, etc.

El proyecto también contempla el uso de herramientas de desarrollo colaborativo como Git y GitHub, las cuales permiten llevar un control de versiones del código y evidenciar la evolución del trabajo a lo largo del avance del desafío. De esta forma, cada desarrollo realizado en el proyecto

queda registrado mediante commits, facilitando tanto la organización del trabajo en equipo como el seguimiento del proceso de construcción del programa.

A continuación, se explicará el análisis del desafío.

Análisis

Primero es necesario tener en cuenta los elementos principales que conforman el funcionamiento del tetris y las operaciones que el programa debe realizar, este análisis buscará descomponer el problema en partes más pequeñas que posteriormente serán implementadas en el programa.

Un elemento fundamental del juego son las piezas, las cuales están formadas por bloques diferentes ya dictaminados en el documento. Cada pieza posee diferentes orientaciones dependiendo de su rotación. Durante el desarrollo del juego las piezas aparecen en la parte superior del tablero no de forma inmediata, sino, una parte de ellas y el jugador puede moverlas hacia abajo o a la izquierda, hacia la derecha o rotarlas, a diferencia del tetris tradicional, las piezas no descenderán automáticamente con el paso del tiempo, sino únicamente cuando el jugador lo indique mediante un comando, estas piezas serán generadas de manera aleatoria mediante una biblioteca de generación de números aleatorios.

El programa debe permitir al usuario definir el tamaño del tablero al inicio del programa. Como restricción del desafío, el ancho del tablero debe ser un múltiplo de 8, lo cual facilita el posible uso de representaciones binarias y operaciones bit a bit para optimizar el manejo de la información. Por otra parte, la altura del tablero es escogida a disposición del jugador.

El tablero se mostrará en la consola utilizando caracteres para representar los espacios vacíos y los bloques ocupados. Después de cada acción del jugador, el tablero deberá actualizarse y mostrarse nuevamente para reflejar el estado actual del juego.

Cuando una pieza alcanza la parte inferior del tablero o entra en contacto con otra pieza, esta quedará fija en el tablero. Y luego se generará una nueva pieza en la parte superior para continuar el juego.

El programa también deberá detectar cuando una fila del tablero esté completamente llena. Al darse esa situación, la línea deberá eliminarse y todas las filas superiores deberán desplazarse hacia abajo una posición. Este proceso permite liberar espacio en el tablero y continuar la partida.

El juego finalizará cuando una nueva pieza no pueda colocarse en la parte superior del tablero debido a que el espacio ya se encuentra ocupado. En ese momento se considerará que el tablero está lleno y la partida habrá terminado.

A continuación, se explicará el diseño del programa.

Diseño

Entre lo diseñado y planteado se busca implementar dentro del tetris estará basado en una estructura de programación modular, donde este se dividirá en distintas funciones que se harán cargo de tareas específicas. Esto hace que sea más fácil el desarrollo del proyecto y permite tener un código organizado y claro.

Primero, se va a implementar funciones de validación, que serán encargadas de verificar que las entradas del usuario sean correctas. Una de ellas comprobará que los caracteres que se

ingresan sean comandos válidos para controlar el movimiento de las piezas dentro del tablero. Otra función estará destinada a validar los valores iniciales del tablero, verificando que el ancho ingresado sea un múltiplo de 8 y que la altura sea igual o mayor a 8 , se puede considerar representaciones basadas en bits de tal forma que se pueda optimizar ciertas operaciones como la verificación de ocupación o la detección de líneas completas.

En lo que se trata de la manipulación de las piezas, se propone el usar funciones que se encarguen de gestionar su movimiento y rotación. Para hacer más sencilla la rotación de las piezas, cada figura se representará dentro de un espacio cuadrado que delimita su espacio máximo de rotación. Por ejemplo, si una pieza ocupa 3 unidades de ancho, se realizará su rotación dentro de un espacio de 3x3. De la misma forma, la pieza en forma de “I”, que tiene 4 unidades de longitud, estará en un espacio 4x4 en el que podremos gestionar el tipo de rotaciones posibles sobre ella.

El sistema también deberá contar con una función que se encargue de detectar colisiones, la cual verificará si una pieza ha llegado al límite inferior del tablero o si ha chocado con otra pieza que ya estuviera situada en el tablero. En cuanto esto ocurra, la pieza se fijará en su posición actual y sus bloques pasarán a formar parte permanente del tablero.

También, el programa hará uso de las funciones para la visualización del tablero en pantalla, la generación de piezas aleatorias y el control del flujo general del juego. El ciclo principal del programa se encargará de recibir las acciones del jugador, y de actualizar el estado del tablero, verificar las colisiones y determinar si el juego debe parar o continuar

Finalmente, el programa deberá incluir una función que compruebe y elimine líneas completas en el tablero. Esta función analiza cada fila para comprobar si todos sus espacios están

ocupados. Cuando esto pase, la línea será eliminada y las filas superiores bajan una posición, lo que permitirá que el juego continúe.

La combinación de las funciones anteriores permite construir una estructura organizada en el programa, de manera que cada módulo se encargue de una parte concreta del funcionamiento del juego. Este diseño servirá como base para la implementación del sistema utilizando el lenguaje C++ con el uso de operaciones bit a bit que permiten la representación y manipulación de las estructuras del juego de forma eficiente, como es requerido por el docente.

Esquema de Tareas

Tarea	**Responsable**	**Orden de desarrollo**
Representación del tablero en bits	Jean Garces	1
Generación aleatoria de piezas	Jean Garces	2
Validación de entrada del usuario	Mateo Arbeláez	3
Movimiento hacia abajo	Jean Garces	4
Detección de colisiones	Jean Garces	5
Movimiento lateral (izquierda y derecha)	Mateo Arbeláez	6
Rotación de piezas	Mateo Arbeláez	7
Game Over	Jean Garces	8
Ciclo principal del juego (main.cpp)	Ambos	9
Integración y pruebas	Jean Garces	10

Algoritmos implementados

En esta sección se describe los algoritmos más importantes desarrollados durante la implementación del juego. El proyecto se dividió de forma modular, cada parte era responsable de una sección distinta de cómo se ejecuta el juego. La mayoría de los algoritmos están diseñados para funcionar a nivel de bits, por lo tanto, los elementos de los mismos se representan de forma eficiente en la memoria y los cálculos que se realizan comúnmente se ejecutan rápidamente.

Representación del tablero en bits:

El tablero se almacena como un arreglo dinámico de bytes. Cada byte representa 8 celdas consecutivas de una fila, donde cada bit indica si la celda está ocupada (1) o libre (0). Esto se puede lograr gracias a que el ancho del tablero siempre es múltiplo de 8, lo que garantiza que cada fila cabe exactamente en un número entero de bytes.

¿Cómo se estructura en memoria?

El tablero es un arreglo de punteros (unsigned char**) donde cada posición apunta a una fila. Cada fila es un arreglo de unsigned char de tamaño anchoTablero / 8. Toda esta memoria se reserva dinámicamente con new porque el tamaño del tablero lo decide el usuario al inicio.

Para acceder a una celda específica se calcula:

- El índice del byte: columna / 8
- El índice del bit dentro del byte: columna % 8

Luego se utilizan operaciones bit a bit (AND, OR y complemento) para consultar, activar o desactivar el bit correspondiente.

¿Qué tipo de dato se usó y por qué?

Se usó unsigned char porque es el tipo más pequeño disponible con exactamente 8 bits, sin bit de signo que pueda complicar las operaciones de desplazamiento. Para el ancho y alto se usó unsigned int, ya que son valores positivos que nunca serán negativos.

¿Por qué es eficiente?

En lugar de usar un int o bool por celda, cada celda ocupa exactamente 1 bit. Por ejemplo, un tablero de 16×20 ocupa solo 40 bytes en lugar de 320 con bool o 1280 con int. Además, operaciones como verificar si una fila está completa se reducen a comparar cada byte con 0xFF, esto reduce significativamente el consumo de memoria y mejora la eficiencia general del programa.

Impresión del tablero:

¿Qué hace?

La función imprimirTablero recorre el tablero fila por fila y celda por celda, mostrando en consola el estado actual del juego. Para cada celda se imprime:

- # si la celda está ocupada (bit en 1)
- . si la celda está libre (bit en 0)

Además, se dibujan bordes laterales y superior/inferior con los caracteres +, - y | para que la presentación sea visualmente clara y fácil de interpretar.

¿Cómo funciona internamente?

Para cada celda, la función llama a obtenerCelda, que se encarga de leer el bit correspondiente mediante el algoritmo de acceso por bits (calculando el byte y la máscara necesaria). La función imprimirTablero no recorre los bits directamente ni maneja las operaciones bit a bit, sino que delega esa responsabilidad en obtenerCelda. Esto mantiene el código organizado, modular y facilita su mantenimiento.

¿Por qué es importante?

Es la única función que le muestra al jugador el estado actual del tablero. Se invoca:

- Al inicio de cada turno
- Después de cada movimiento
- Al finalizar la partida, cuando ocurre el Game Over

De esta forma, el jugador siempre tiene una visual clara del progreso de la partida.

Detección de colisiones:

¿Qué hace?

La función colisionValida simula el movimiento de la pieza antes de ejecutarlo. Recibe como parámetros la posición futura donde querría ir la pieza (nuevaCol y nuevaFila) y calcula dónde quedaría cada bloque si la pieza se moviera ahí. Retorna 1 si ningún bloque choca con nada (movimiento válido), o 0 si alguno chocaría (movimiento inválido).

¿Cómo funciona internamente?

Recorre bit por bit cada celda de la pieza actual. Para cada bit en 1 (celda activa de la pieza), calcula su posición futura en el tablero sumando la posición hipotética a las coordenadas locales de la caja:

colReal = nuevaCol + col

filaReal = nuevaFila + fila

Luego verifica cuatro condiciones:

1. Borde izquierdo → colReal < 0
2. Borde derecho → colReal >= anchoTablero
3. Borde inferior → filaReal >= altoTablero
4. Colisión con otra pieza → obtenerCelda(colReal, filaReal) retorna 1

Si cualquiera de las cuatro se cumple, retorna 0 inmediatamente (movimiento inválido). Si ninguna se cumple, retorna 1 (movimiento válido).

¿Por qué la condición filaReal >= 0 antes de consultar el tablero?

Es importante destacar que no se verifica el borde superior (filaReal < 0) como condición de colisión. Esto se debe a que cuando la pieza está asomada desde arriba (recién generada), algunas filas de su caja tienen filaReal negativa, lo que significa que están fuera del tablero. Consultar el tablero con una fila negativa causaría un acceso a memoria inválido. Por esta razón, antes de llamar a obtenerCelda se debe asegurar que filaReal >= 0; de lo contrario, se omite esa verificación, permitiendo que la pieza pueda ingresar gradualmente desde la parte superior.

¿Dónde se usa?

La función se llama antes de cada movimiento en:

- bajarPieza — para verificar si se puede desplazar hacia abajo
- moverPieza — para validar desplazamientos laterales
- rotarPieza — para comprobar si la rotación es viable
- generarPieza — para detectar el Game Over al intentar colocar una nueva pieza

De esta forma, se garantiza que todas las acciones del jugador y la lógica automática del juego respeten los límites del tablero y las colisiones con las piezas ya fijadas.

Generación aleatoria de las piezas

¿Qué hace?

La función generarPieza selecciona aleatoriamente una de las piezas disponibles, carga sus bits en piezaActual, la posiciona en la parte superior central del tablero y la dibuja. También verifica si el juego debe terminar. Se llama al inicio del juego y cada vez que una pieza aterriza en bajarPieza.

¿Cómo se representan las piezas?

Cada pieza se define como un arreglo de MAX_CAJA bytes, donde cada byte representa una fila de la caja cuadrada que contiene a la pieza. Los bits de cada byte indican qué celdas están ocupadas, empezando desde la izquierda (bit más significativo).

Por ejemplo, la pieza **L** se representa como:

Forma	**Bits**
1 0 0	10000000
1 0 0	10000000
1 1 0	11000000
0 0 0	00000000

Todas las piezas se almacenan en un arreglo formas[NUM_PIEZAS][MAX_CAJA] y sus tamaños de caja en tamaños[NUM_PIEZAS].

¿Cómo funciona la aleatoriedad?

Se usa rand() % NUM_PIEZAS para obtener un número entre 0 y 4 que corresponde a una de las 5 piezas disponibles. La biblioteca utilizada es <cstdlib>.

¿Cómo aparece la pieza en el tablero?

La pieza aparece centrada horizontalmente mediante:

colPieza = (anchoTablero / 2) - (tamanioCaja / 2)

Para que solo la última fila visible aparezca en la fila 0 del tablero, filaPieza se inicializa con un valor negativo:

filaPieza = -ultimaFila

Donde ultimaFila es el índice de la última fila de la caja que tiene al menos un bit en 1. De esta forma, las filas superiores de la caja quedan fuera del tablero y solo la última fila visible asoma desde arriba, permitiendo que la pieza ingrese gradualmente a medida que desciende.

¿Cómo se detecta el Game Over aquí?

Antes de dibujar la pieza se llama a colisionValida con la posición inicial. Si retorna 0, significa que no hay espacio para colocar la nueva pieza porque las celdas donde debería aparecer ya están ocupadas por piezas fijadas. En ese caso, se activa gameOver = true.

¿La función evalúa todas las posiciones posibles?

No. generarPieza solo evalúa una única posición inicial (la centrada en la parte superior). Si esa posición es inválida, se declara Game Over. No explora posiciones alternativas para colocar la pieza; simplemente asume que, si no cabe en la posición de entrada estándar, el juego ha terminado.

Movimiento lateral de piezas:

¿Qué hace?

La función moverPieza mueve la pieza una posición hacia la izquierda o derecha según el valor del parámetro direccion. Recibe -1 para mover a la izquierda y 1 para mover a la derecha. Retorna 1 si el movimiento fue exitoso o 0 si no fue posible.

¿Cómo funciona internamente?

El proceso consta de tres pasos:

1. Borrar la pieza de su posición actual en el tablero usando borrarPiezaTablero, que limpia bit por bit todos los bloques de la pieza.

2. Verificar si puede moverse llamando a colisionValida con la nueva columna hipotética:

colisionValida(colPieza + direccion, filaPieza)

3. Actualizar según el resultado:

   - Si retorna 1 → se actualiza colPieza sumándole direccion y se redibuja la pieza con dibujarPiezaTablero
   - Si retorna 0 → la pieza no se mueve pero se redibuja en su posición original

¿Por qué se borra la pieza antes de verificar la colisión?

Porque si no se borrara, los bits de la pieza actual en el tablero interferirían con la verificación de colisión. La función obtenerCelda vería los bits de la propia pieza como obstáculos (celdas ocupadas) y podría bloquear movimientos válidos, impidiendo que la pieza se desplace incluso cuando hay espacio libre en la dirección deseada. Borrar la pieza temporalmente permite verificar únicamente la colisión con las piezas fijadas.

¿Dónde se usa?

Se llama desde main.cpp cuando el usuario presiona:

- a → mover a la izquierda
- d → mover a la derecha

Descenso de piezas

¿Qué hace?

La función bajarPieza intenta mover la pieza una posición hacia abajo. Retorna 1 si la pieza bajó exitosamente o 0 si la pieza aterrizó y se generó una nueva.

¿Cómo funciona internamente?

El proceso consta de tres pasos:

1. Borrar la pieza de su posición actual en el tablero usando borrarPiezaTablero.
2. Verificar si puede bajar llamando a colisionValida con la nueva fila hipotética:

colisionValida(colPieza, filaPieza + 1)

3. Actualizar según el resultado:
    - Si retorna 1 → se incrementa filaPieza en 1 y se redibuja la pieza con dibujarPiezaTablero
    - Si retorna 0 → la pieza aterrizó, se redibuja en su posición actual para fijarla permanentemente, se limpian las filas completas y se genera una nueva pieza

¿Qué diferencia tiene con el movimiento lateral?

La diferencia principal es lo que ocurre cuando el movimiento no es válido:

Movimiento lateral (moverPieza)	Descenso (bajarPieza)
Si no es válido, la pieza simplemente no se mueve	Si no es válido, la pieza se fija permanentemente en el tablero
Se redibuja la pieza en su posición original	Se genera una nueva pieza y se continúa el juego

En el descenso, cuando colisionValida retorna 0, significa que la pieza llegó al fondo o chocó con otra pieza, por lo que debe quedar fijada en esa posición.

¿Por qué se llama a limpiarFilasCompletas aquí?

Porque es el momento justo después de que una pieza aterriza cuando pueden aparecer filas completas. Si al fijarse la pieza completa una o más filas, estas se eliminan inmediatamente antes de generar la siguiente pieza. Esto garantiza que:

- Las filas completas no persistan en el tablero
- Las piezas superiores desciendan correctamente tras la eliminación
- La nueva pieza se genere en un tablero actualizado

¿Dónde se usa?

Se llama desde main.cpp cuando el usuario presiona s para hacer descender la pieza manualmente. También suele invocarse en el bucle principal del juego para el descenso automático por tiempo.

Dibujo y borrado de piezas

¿Qué hacen?

dibujarPiezaTablero y borrarPiezaTablero son dos funciones auxiliares que se encargan de marcar y desmarcar los bits de la pieza activa en el tablero. Convierten la representación interna de la pieza en modificaciones reales sobre el tablero:

- Dibujar **(**dibujarPiezaTablero**)**: activa los bits correspondientes en el tablero
- Borrar **(**borrarPiezaTablero**)**: desactiva dichos bits antes de mover o rotar la pieza

Son la base del movimiento, la rotación y la generación de piezas, ya que todo desplazamiento implica primero borrar la pieza de su posición actual y luego dibujarla en la nueva, evitando así que la pieza deje rastros al desplazarse.

¿Cómo funciona borrarPiezaTablero?

Recorre bit por bit la caja de la pieza actual. Para cada bit en 1 (celda activa de la pieza), calcula su posición real en el tablero sumando la posición actual de la pieza a las coordenadas locales:

colReal = colPieza + col

filaReal = filaPieza + fila

Luego llama a limpiarCelda para poner ese bit en 0. Solo actúa sobre celdas cuya filaReal >= 0 para evitar acceder a posiciones fuera del tablero (cuando la pieza está parcialmente fuera por arriba).

¿Cómo funciona dibujarPiezaTablero?

Hace exactamente lo mismo pero en sentido contrario. Para cada bit en 1 de la pieza, calcula su posición real y llama a setCelda para poner ese bit en 1 en el tablero. Al igual que borrarPiezaTablero, solo actúa sobre celdas cuya filaReal >= 0.

¿Por qué son funciones separadas?

Porque se usan en momentos diferentes y con propósitos distintos:

Función	Cuándo se usa
borrarPiezaTablero	Antes de cualquier movimiento o rotación
dibujarPiezaTablero	Después de cualquier movimiento o rotación

Separarlas hace el código más modular, evita repetir el mismo ciclo de recorrido en cada función de movimiento y facilita el mantenimiento.

¿Dónde se usan?

Se llaman en:

- bajarPieza — descenso vertical
- moverPieza — movimiento lateral
- rotarPieza — rotación de la pieza

En todos estos casos, el patrón es el mismo: borrar, verificar, actualizar posición, dibujar.

Rotación de piezas

¿Qué hace?

La función rotarPieza rota la pieza activa 90 grados en sentido horario. Si la rotación no es válida por colisión o por salirse del tablero, la pieza vuelve a su forma original. La rotación se realiza en varios pasos: se copia la pieza actual como respaldo, se convierte su representación en una matriz binaria auxiliar, se aplica una transformación de rotación de 90 grados, se reconstruye la pieza en formato de bits y se verifica si la nueva orientación produce colisión.

¿Cómo funciona internamente?

El proceso consta de cuatro pasos:

Paso 1: Guardar la pieza original antes de rotar, se copia piezaActual en un arreglo temporal piezaOriginal por si la rotación falla y hay que restaurarla.

Paso 2: Convertir bits a matriz los bits de la pieza se convierten a una matriz de 0s y 1s para poder aplicar la fórmula de rotación más fácilmente:

matriz[fila][col] = 1 si el bit estaba en 1

matriz[fila][col] = 0 si el bit estaba en 0

Paso 3: Aplicar la rotación matemática se aplica la fórmula de rotación 90 grados en sentido horario sobre la matriz:

nuevaFila = col

nuevaCol = (tamanioCaja - 1) - fila

Esto significa que lo que estaba en la columna “col” pasa a la fila “col”, y lo que estaba en la fila “fila” pasa a la columna “tamanioCaja - 1 – fila”, invirtiendo el orden de izquierda a derecha.

Paso 4: Convertir matriz rotada a bits la matriz rotada se convierte nuevamente a bytes usando máscaras, reconstruyendo piezaActual con la nueva forma.

¿Cómo se verifica si la rotación es válida?

Después de calcular la nueva forma:

1. Se borra la pieza del tablero con borrarPiezaTablero
2. Se asigna la nueva forma a piezaActual
3. Se llama a colisionValida con la posición actual (colPieza, filaPieza)

Si retorna 1 → la rotación es válida: se dibuja la pieza rotada con dibujarPiezaTablero
Si retorna 0 → la rotación es inválida: se restaura piezaOriginal y se redibuja la forma anterior

¿Por qué se calcula en tiempo real y no se guardan las rotaciones?

Porque guardar todas las rotaciones en arreglos pregrabados consumiría más memoria y no aplicaría los conceptos de operaciones a nivel de bits que exige el desafío. Calcularla matemáticamente demuestra el dominio de la manipulación de matrices y bits, manteniendo el

código más eficiente en memoria y alineado con el enfoque de bajo nivel utilizado en el resto de la implementación.

¿Dónde se usa?

Se llama desde main.cpp cuando el usuario presiona w para rotar la pieza activa.

Detección de filas completas

¿Qué hace?

La función filaCompleta verifica si una fila específica del tablero tiene todas sus celdas ocupadas. Retorna 1 si la fila está completa o 0 si tiene al menos una celda vacía. Una fila se considera completa cuando todos sus bytes contienen el valor 255 (11111111 en binario), lo que indica que todas sus celdas están ocupadas.

¿Cómo funciona internamente?

Recorre todos los bytes de la fila y los compara con 0xFF:

- 0xFF = 11111111 en binario
- Si todos los bytes de la fila son iguales a 0xFF, significa que todos los bits están en 1, es decir, todas las celdas están ocupadas y la fila está completa
- Si algún byte es diferente de 0xFF, la fila tiene al menos una celda vacía y retorna 0 inmediatamente

¿Por qué se compara con 0xFF y no bit por bit?

Porque es mucho más eficiente. En lugar de verificar 8 bits individualmente, se verifica un byte completo en una sola comparación. Por ejemplo:

Ancho del tablero	Comparaciones byte a byte	Comparaciones bit a bit
16 columnas	2 comparaciones	16 comparaciones
32 columnas	4 comparaciones	32 comparaciones

Esto reduce significativamente el tiempo de verificación, especialmente en tableros grandes.

¿Por qué esto es posible?

Precisamente porque el ancho del tablero es siempre múltiplo de 8. Esto garantiza que:

- Cada fila está compuesta por un número entero de bytes
- No hay bits sobrantes ni parciales en los bytes de cada fila
- Si todos los bytes son 0xFF, realmente todas las celdas están ocupadas sin excepción

Si el ancho no fuera múltiplo de 8, habría bits residuales que complicarían esta verificación.

¿Dónde se usa?

Se llama desde limpiarFilasCompletas para revisar cada fila del tablero después de que una pieza aterriza, identificando qué filas deben eliminarse.

Eliminación de filas

¿Qué hace?

La función eliminarFila elimina una fila específica del tablero, baja todas las filas superiores una posición y crea una nueva fila vacía en la parte superior. Cuando se detecta una fila completa, se libera la memoria asociada a dicha fila, se desplazan hacia abajo las referencias a las filas superiores y se crea una nueva fila vacía en la parte superior del tablero.

¿Cómo funciona internamente?

El proceso consta de tres pasos:

Paso 1: Liberar la fila completa se libera la memoria de la fila que está llena usando delete[]:

```
delete[] celdas[fila]
```

Paso 2: Bajar los punteros en lugar de copiar el contenido de cada fila hacia abajo, se mueven directamente los punteros del arreglo:

```
celdas[i] = celdas[i - 1]
```

Esto recorre desde la fila eliminada hacia arriba, haciendo que cada puntero apunte a la fila que estaba encima. Es mucho más eficiente porque solo se reasignan direcciones de memoria en lugar de copiar bytes.

Paso 3: Crear nueva fila vacía en la cima se reserva memoria para una nueva fila en la posición 0 y se inicializa en 0:

celdas[0] = new unsigned char[bytesPorFila]

Todos sus bytes se ponen en 0, representando una fila completamente vacía.

¿Por qué se mueven punteros en lugar de copiar contenido?

Porque mover punteros es una operación de tiempo constante, independiente del ancho del tablero. Copiar contenido byte por byte dependería del ancho y sería menos eficiente. Esto es posible gracias a que el tablero usa memoria dinámica con unsigned char**, donde cada fila es un bloque independiente.

Este enfoque es especialmente eficiente cuando el ancho del tablero es grande, ya que evita copiar grandes cantidades de datos.

¿Dónde se usa?

Se llama desde limpiarFilasCompletas cada vez que detecta una fila completa. Al eliminarlas de arriba hacia abajo, se mantiene la integridad del tablero.

Validación de entrada del usuario

El módulo comprobacion.cpp contiene dos funciones encargadas de validar las entradas del usuario: pedirLetra, que, valida los comandos del juego, y pedirDimensiones, que valida el tamaño del tablero al inicio. El programa solicita comandos mediante teclado para controlar el juego, aceptando solo las teclas definidas para movimiento, rotación o salida. Si el usuario introduce un valor inválido, el sistema muestra un mensaje de error y vuelve a solicitar la entrada, garantizando así un control robusto del juego.

¿Qué hace pedirLetra?

Solicita al usuario una letra y verifica que sea uno de los cinco comandos válidos del juego: a, s, d, w o q. Si el usuario ingresa cualquier otro carácter, muestra un mensaje de error y vuelve a pedir la entrada.

¿Cómo funciona internamente?

Usa un bloque try/catch dentro de un while(true):

1. Se solicita un carácter al usuario
2. Si la letra ingresada no es válida, se lanza un error con throw
3. El bloque catch atrapa el error, muestra el mensaje correspondiente y limpia el buffer de entrada con cin.clear() y cin.ignore() para evitar que entradas inválidas bloqueen el programa
4. El bucle continúa hasta que se ingresa un comando válido

¿Qué hace pedirDimensiones?

Solicita al usuario el alto y ancho del tablero y valida que:

- La altura sea mínimo 8
- El ancho sea mínimo 8 y múltiplo de 8

¿Cómo funciona internamente?

Lee los valores en una variable int temporal con signo en lugar de directamente en unsigned int. Esto permite detectar números negativos antes de asignarlos, ya que un número negativo

asignado directamente a unsigned int se convertiría en un número gigante positivo por desbordamiento. Si algún valor es inválido, se lanza un error con throw que es atrapado por catch.

¿Por qué se reciben los parámetros por referencia (&)?

Porque las variables altoTablero y anchoTablero están definidas en tablero.cpp. Al pasarlas por referencia, la función trabaja directamente sobre esas variables sin hacer copias, modificándolas desde comprobacion.cpp. Esto permite:

- Modificar los valores originales sin necesidad de retornarlos
- Evitar copias innecesarias de datos
- Mantener una interfaz clara y modular

¿Dónde se usan?

Función	Llamada desde
pedirLetra	Cada turno del ciclo principal en main.cpp
pedirDimensiones	inicializarTablero en tablero.cpp

Problemas de desarrollo afrontado

Durante la implementación del proyecto, se dieron varios problemas que estaban relacionados con enfoques de tipo técnicos y organizativos. Algunos de los problemas estaban relacionados con el desconocimiento del QT, y otros era tales como la inconsistencia del diseño

conceptual inicial, que solo se reflejaron mientras se integraban los módulos en un solo sistema. Además, algunos de los errores se debieron al nivel de complejidad total para trabajar con aplicaciones a nivel de bits, donde un error en una o más áreas puede provocar el error general. En la tabla a continuación, vea ejemplos contemporáneos de problemas en la implementación.

1. El programa no permitía ingresar datos por consola en Qt

El primer problema encontrado fue que al ejecutar el programa en Qt Creator, el cin no esperaba la entrada del usuario y el programa terminaba instantáneamente o entraba en un loop infinito. Esto ocurría porque Qt Creator por defecto no abre una terminal interactiva real, sino que ejecuta el programa en un proceso interno que no permite entrada del usuario. La solución fue activar la opción Run in terminal en la configuración de ejecución del proyecto en Qt Creator, lo cual hace que el programa corra en una terminal externa donde cin funciona correctamente.

2. Variables globales no inicializadas causaban loop infinito

Al inicio del desarrollo, las variables globales anchoTablero y altoTablero no se inicializaban correctamente en 0, lo que causaba que el bucle de validación de entrada corriera infinitamente sin dejar al usuario escribir nada. La solución fue inicializar explícitamente todas las variables globales en su definición en tablero.cpp y usar una variable int temporal con signo para leer los valores antes de asignarlos a unsigned int.

3. Aparición de bits fantasma en el tablero

Durante las pruebas se observó que al mover o rotar piezas aparecían bits que no correspondían a ninguna pieza activa. Esto ocurría porque las funciones de movimiento no borraban correctamente todos los bits de la pieza antes de redibujarla en su nueva posición. La solución fue crear las funciones auxiliares borrarPiezaTablero y dibujarPiezaTablero que se

encargan de borrar y dibujar la pieza completa de forma consistente, y usarlas en todas las funciones de movimiento y rotación.

4. Inconsistencia entre generarPieza y las funciones de movimiento

Al principio generarPieza dibujaba solo la última fila visible de la pieza en la fila 0 del tablero, pero las funciones borrarPiezaTablero y dibujarPiezaTablero trabajaban con la pieza completa usando filaPieza. Esto causaba que al hacer el primer movimiento después de generar una pieza, algunos bits quedaran sin borrar y aparecieran como bits fantasma. La solución fue inicializar filaPieza con un valor negativo igual a -ultimaFila, de modo que la posición de la pieza fuera consistente desde el principio y borrarPiezaTablero pudiera borrarla correctamente.

5. Problema con la rotación de piezas

La función de rotación presentaba dos problemas. El primero era que no borraba la pieza antes de aplicar la rotación, lo que dejaba bits de la forma anterior en el tablero. El segundo era que verificaba la colisión con la pieza original en lugar de con la pieza ya rotada, lo que podía permitir rotaciones inválidas. La solución fue reorganizar el orden de operaciones: primero borrar la pieza con borrarPiezaTablero, luego asignar la nueva forma rotada a piezaActual y finalmente verificar la colisión con la forma ya rotada antes de dibujarla.

6. Organización inicial del proyecto

Al inicio del desarrollo se crearon proyectos Qt separados para cada módulo (tablero y piezas), lo que generó confusión y duplicación de archivos. Se decidió unificar todo en un solo proyecto Qt llamado tetris donde todos los módulos conviven en la misma carpeta, con un único CMakeLists.txt que incluye todos los archivos. Esto simplificó la compilación y el manejo del repositorio.

7. Commits incorrectos en el repositorio

Durante el proceso de desarrollo hubo casos donde los mensajes de los commits quedaron incompletos o mal escritos. Esto se resolvió usando el comando git commit --amend para corregir el mensaje del último commit antes de hacer git push, y teniendo más cuidado al escribir los mensajes de commit posteriores.

Evolución de la solución y consideraciones para tener en cuenta en la implementación

El desarrollo del proyecto no siguió un camino lineal, sino que fue evolucionando a medida que se implementaban los módulos y se descubrían nuevos problemas. A continuación, se describe cómo fue avanzando la solución desde el diseño inicial hasta la implementación final.

Etapa 1: Diseño y planificación Antes de escribir cualquier línea de código se realizó un análisis del problema para identificar los elementos principales del juego y cómo representarlos eficientemente, esto es principalmente lo mostrado en el informe preliminar. Se decidió usar operaciones a nivel de bits para representar el tablero y las piezas, y se definió una estructura modular donde cada archivo se encargaría de una parte específica del juego. También se creó el repositorio en GitHub y la estructura inicial del proyecto en Qt.

Etapa 2: Implementación del tablero El primer módulo implementado fue el tablero, ya que es la base sobre la cual se construye todo el juego. Se implementaron las funciones de inicialización, manipulación de celdas con operaciones de bits, detección y eliminación de filas completas e impresión por consola. Durante esta etapa se descubrió el problema con Qt Creator y el cin, que se resolvió activando la opción Run in terminal.

Etapa 3: Generación de piezas Una vez el tablero funcionaba correctamente se implementó la generación aleatoria de piezas. Se definieron las 5 piezas en bits dentro de cajas cuadradas y se implementó la lógica para que cada pieza apareciera centrada en la parte superior del tablero mostrando solo la última fila visible. En esta etapa surgió la inconsistencia entre generarPieza y las funciones de movimiento que se resolvió con el uso de filaPieza negativo.

Etapa 4: Movimiento y colisiones Se implementaron las funciones de detección de colisiones, movimiento hacia abajo y movimiento lateral.

Etapa 5: Rotación de piezas Se implementó la rotación matemática de 90 grados usando la fórmula de transposición y reflexión de matrices. Esta fue la etapa más compleja porque requería convertir bits a matriz, aplicar la rotación y reconvertir a bits, además de manejar correctamente el borrado y redibujado de la pieza.

Etapa 6: Validación de entrada y ciclo principal Se implementó la validación de entrada del usuario con pedirLetra y pedirDimensiones, y se integró todo en el ciclo principal del juego en main.cpp. También se implementó la detección del Game Over.

Etapa 7: Integración y pruebas Una vez todos los módulos estaban implementados se integraron en un solo proyecto y se realizaron pruebas exhaustivas para verificar el correcto funcionamiento del juego. Se probaron casos límite como mover piezas hasta los bordes, rotar piezas en espacios reducidos, llenar filas completas y apilar piezas hasta el Game Over.

Consideraciones para tener en cuenta en la implementación

- **El orden de las operaciones es crítico** en las funciones de movimiento y rotación. Siempre se debe borrar la pieza antes de verificar la colisión y dibujarla después. Alterar este orden causa bits fantasma.

- **La consistencia entre módulos es fundamental.** Las variables globales como filaPieza y colPieza son compartidas entre varios archivos, por lo que cualquier cambio en su valor afecta el comportamiento de todas las funciones que las usan.

- **Trabajar con bits requiere especial atención a los tipos de datos.** Mezclar int con unsigned int o unsigned char puede causar comportamientos inesperados, especialmente en operaciones de desplazamiento. Siempre se deben usar casts explícitos cuando sea necesario.

- **Las pruebas deben hacerse módulo por módulo** antes de integrar todo. Probar el tablero solo, luego las piezas solas y finalmente todo junto facilita identificar el origen de los errores.

- **Git debe usarse de forma regular** con commits frecuentes y mensajes descriptivos para evidenciar la evolución del proyecto y facilitar la recuperación ante errores.